

03

Algorithme du gradient

Automne 2026

Techniques d'apprentissage automatique — 420-C74-SF — gr. 10017
Spécialiste en solutions d'intelligence artificielle — M. Swawola, M.Sc.

NOUS ÉCLAIRON.
VOUS BRILLEZ.

FORMATION CONTINUE
ET SERVICES AUX ENTREPRISES



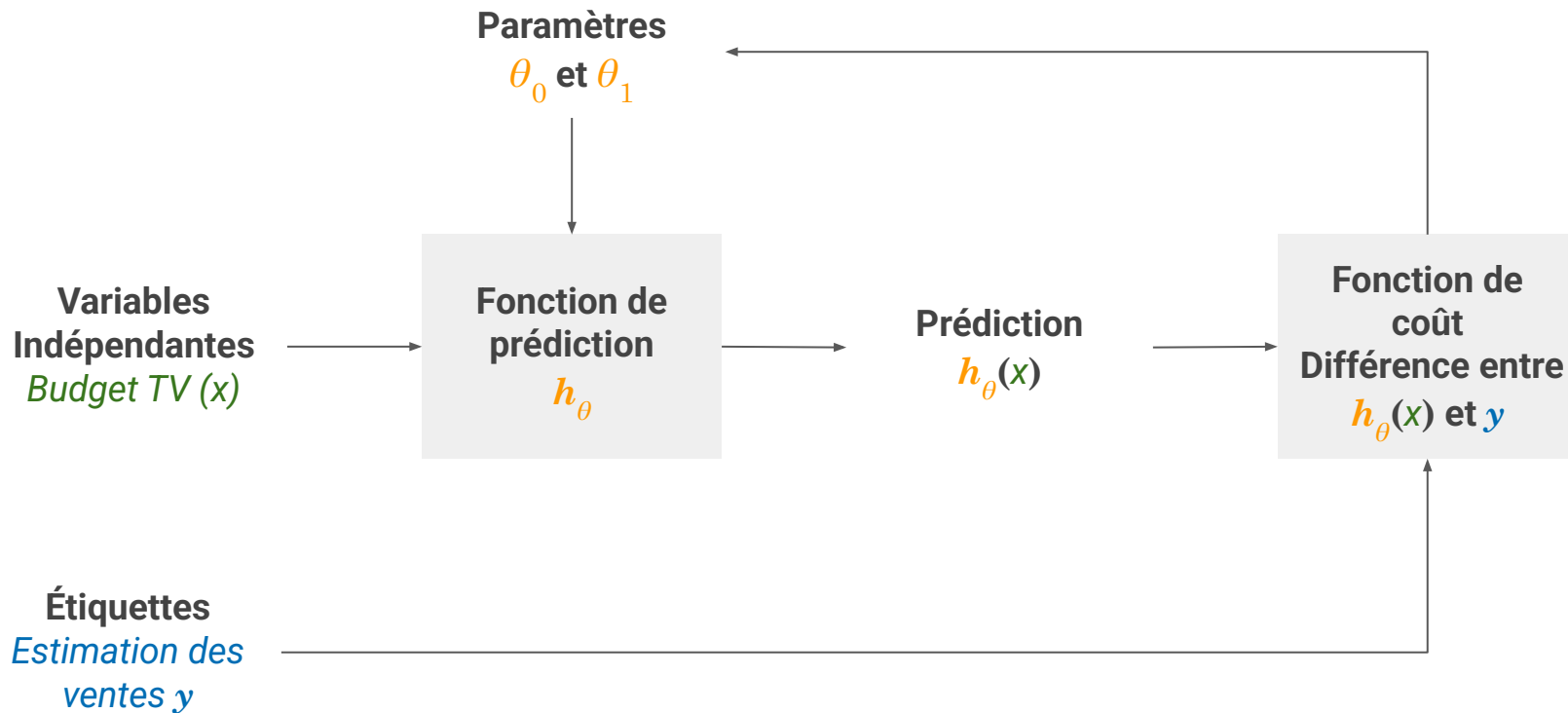
Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- Algorithme du gradient
- Choix du pas
- Régression linéaire simple
- Hyperparamètres
- Lectures et références

Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- Algorithme du gradient
- Choix du pas
- Régression linéaire simple
- Hyperparamètres
- Lectures et références

Représentation du modèle



Quelques éléments de notation

- m → taille du jeu de données
- x → variable d'entrée
- y → variable de sortie
- (x, y) → un **exemple d'entraînement**
- $(x^{(i)}, y^{(i)})$ → $i^{\text{ème}}$ exemple d'entraînement

Budget TV (x)	Ventes (y)
230.1	22.1
44.5	10.4
17.2	9.3
151.5	18.5
...	...

Ci-dessus, un extrait du jeu de données **Advertising**

Question

- Considérons le jeu de données générique ci-contre. $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ représente le $i^{\text{ième}}$ exemple d'entraînement. Que valent les éléments suivants ?
 - $\mathbf{y}^{(3)}$
 - $\mathbf{x}^{(1)}$
 - $\mathbf{m}$

$\mathbf{x}$	$\mathbf{y}$
38.2	7.6
94.2	9.7
177.0	12.8
283.6	25.5
232.1	13.4

Réponse

- Considérons le jeu de données générique ci-contre. $(\mathbf{x}^{(i)}, \mathbf{y}^{(i)})$ représente le $i^{\text{ième}}$ exemple d'entraînement. Que valent les éléments suivants ?
 - $\mathbf{y}^{(3)} = 25.5$
 - $\mathbf{x}^{(1)} = 94.2$
 - $\mathbf{m} = 5$
- Les données sont généralement indexées à partir de zéro (Zero-based numbering)

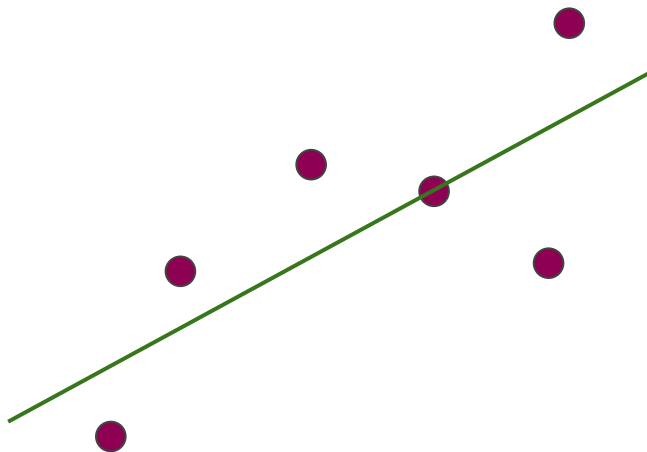
$\mathbf{x}$	$\mathbf{y}$
38.2	7.6
94.2	9.7
177.0	12.8
283.6	25.5
232.1	13.4

Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- **Fonction de coût**
- Algorithme du gradient
- Choix du pas
- Régression linéaire simple
- Hyperparamètres
- Lectures et références

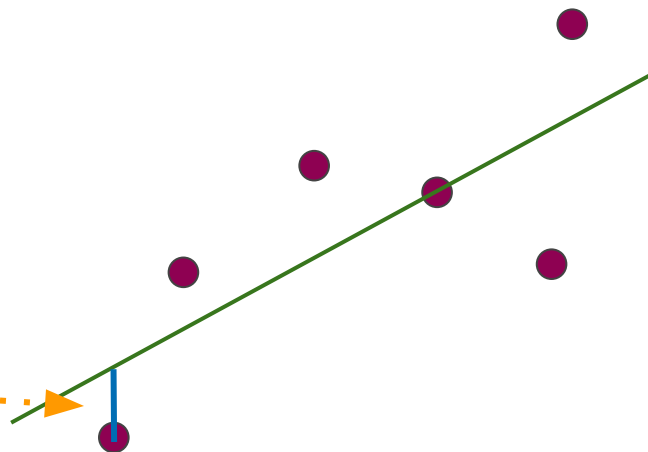
Fonction de coût

- Soit les données et l'hypothèse h_θ ci-contre
- Les paramètres θ_0 et θ_1 sont choisis de manière à minimiser l'erreur entre la donnée y et la prédiction du modèle $h_\theta(x)$ pour cette même donnée



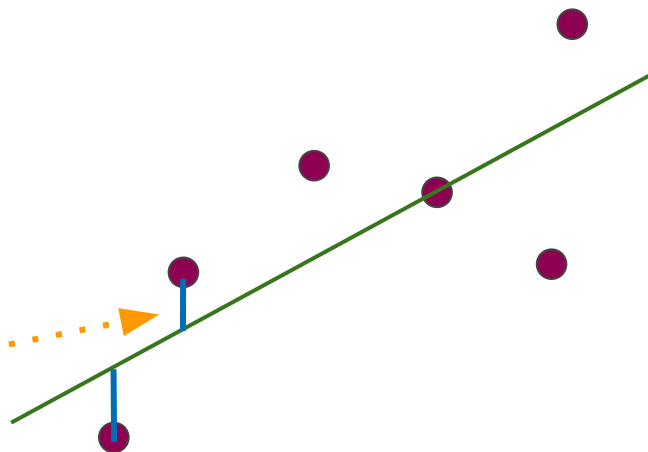
Fonction de coût

- Soit les données et l'hypothèse h_θ ci-contre
- Les paramètres θ_0 et θ_1 sont choisis de manière à minimiser l'erreur entre la donnée y et la prédiction du modèle $h_\theta(x)$ pour cette même donnée
- Erreur = $y^{(0)} - h_\theta(x^{(0)})$



Fonction de coût

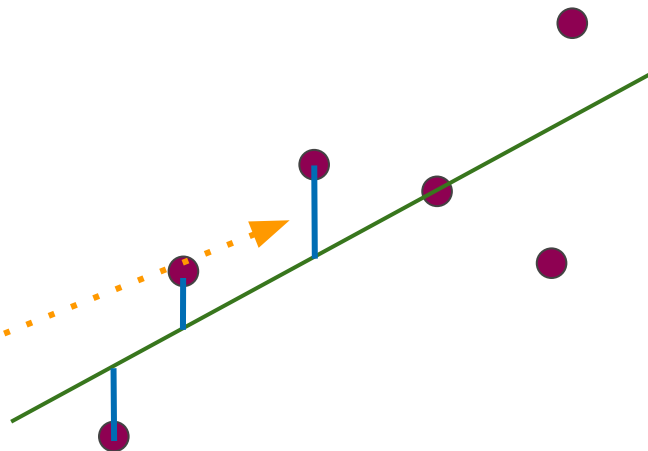
- Soit les données et l'hypothèse h_θ ci-contre
- Les paramètres θ_0 et θ_1 sont choisis de manière à minimiser l'erreur entre la donnée y et la prédiction du modèle $h_\theta(x)$ pour cette même donnée
- Erreur = $y^{(0)} - h_\theta(x^{(0)}) +$
 $y^{(1)} - h_\theta(x^{(1)})$



Fonction de coût

- Soit les données et l'hypothèse h_θ ci-contre
- Les paramètres θ_0 et θ_1 sont choisis de manière à minimiser l'erreur entre la donnée y et la prédiction du modèle $h_\theta(x)$ pour cette même donnée

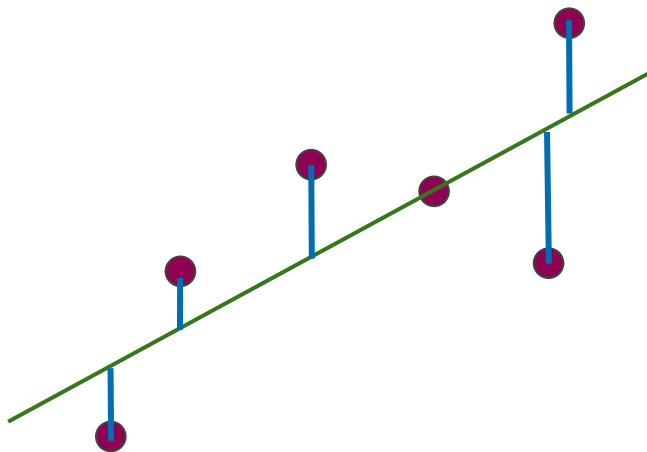
- Erreur = $y^{(0)} - h_\theta(x^{(0)}) +$
 $y^{(1)} - h_\theta(x^{(1)}) +$
 $y^{(2)} - h_\theta(x^{(2)})$



Fonction de coût

- Soit les données et l'hypothèse h_θ ci-contre
- Les paramètres θ_0 et θ_1 sont choisis de manière à minimiser l'erreur entre la donnée y et la prédiction du modèle $h_\theta(x)$ pour cette même donnée

- Erreur = $y^{(0)} - h_\theta(x^{(0)}) +$
 $y^{(1)} - h_\theta(x^{(1)}) +$
 $y^{(2)} - h_\theta(x^{(2)}) +$
 $y^{(3)} - h_\theta(x^{(3)}) +$
 $y^{(4)} - h_\theta(x^{(4)}) +$
 $y^{(5)} - h_\theta(x^{(5)})$



**Mais il n'y a pas un problème de signe ici ?
Les erreurs ne risquent-elles pas de s'annuler ?**

Fonction de coût

■ Erreur = $y^{(0)} - h_{\theta}(x^{(0)}) +$
 $y^{(1)} - h_{\theta}(x^{(1)}) +$
 $y^{(2)} - h_{\theta}(x^{(2)}) +$
 $y^{(3)} - h_{\theta}(x^{(3)}) +$
 $y^{(4)} - h_{\theta}(x^{(4)}) +$
 $y^{(5)} - h_{\theta}(x^{(5)})$



SSE = $(y^{(0)} - h_{\theta}(x^{(0)}))^2 +$
 $(y^{(1)} - h_{\theta}(x^{(1)}))^2 +$
 $(y^{(2)} - h_{\theta}(x^{(2)}))^2 +$
 $(y^{(3)} - h_{\theta}(x^{(3)}))^2 +$
 $(y^{(4)} - h_{\theta}(x^{(4)}))^2 +$
 $(y^{(5)} - h_{\theta}(x^{(5)}))^2$



Somme des erreurs quadratiques

$$\sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Fonction de coût

- Dans la pratique, plutôt que de chercher à minimiser la SSE, nous cherchons à minimiser la moyenne des erreurs quadratiques, ou **erreur quadratique moyenne** (MSE)

$$\text{MSE} = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- Cette quantité que nous allons tenter de minimiser sera notre **fonction de coût / cost function** et sera notée

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \quad \dots \dots \dots \rightarrow \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

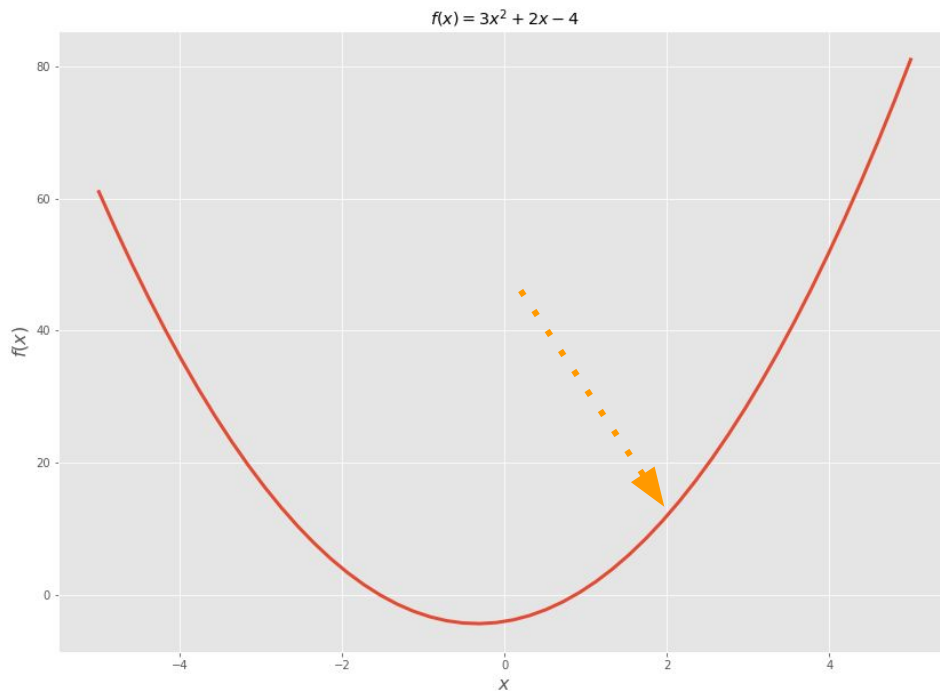
- Nous allons donc avoir besoin d'un **algorithme** pour **apprendre** les paramètres θ_0 et θ_1 de manière à minimiser la fonction de coût

Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- **Algorithme du gradient**
- Choix du pas
- Régression linéaire simple
- Hyperparamètres
- Lectures et références

Question

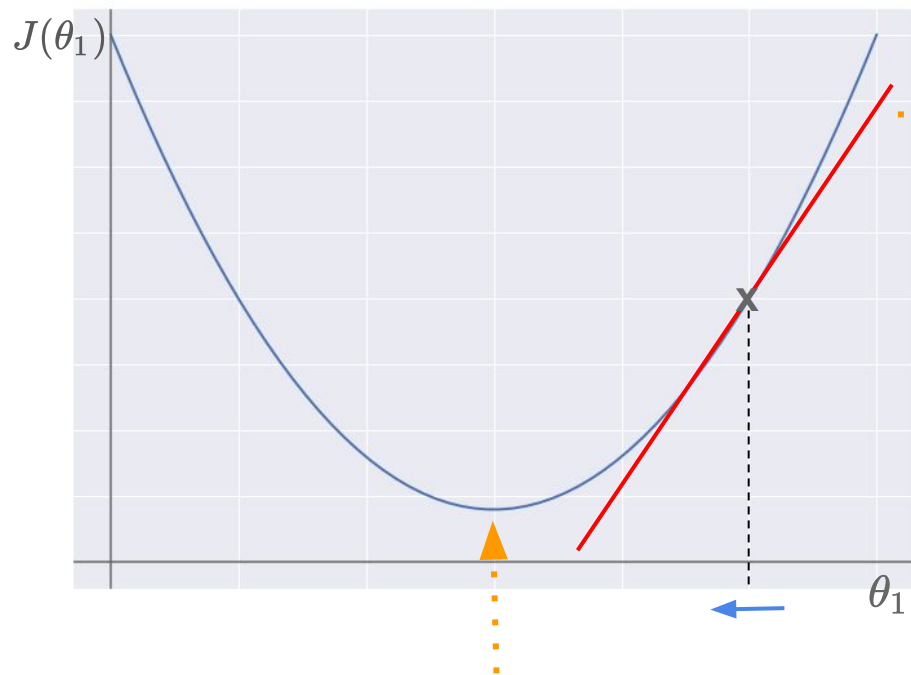
- Quelle est la pente de la fonction $f(x) = 3x^2 + 2x - 4$ au point d'abscisse $x = 2$?



Réponse

- Quelle est la pente de la fonction $f(x) = 3x^2 + 2x - 4$ au point d'abscisse $x = 2$?
- La pente est donnée par la dérivée de $f(x)$ au point $x = 2$
- La dérivée de $f(x)$ est $f'(x) = 6x + 2$
- Donc au point $x = 2$, la pente est $f'(2) = 6 \times 2 + 2 = 14$

Algorithme du gradient



On veut trouver le minimum de $J(\theta_1)$

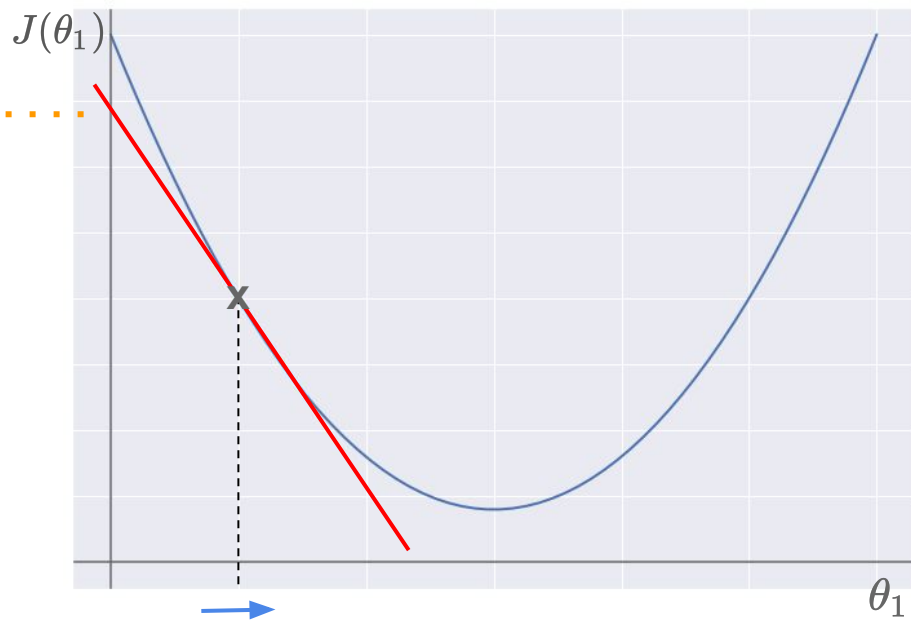
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

$$\theta_1 := \theta_1 - \alpha \times \text{nombre positif}$$

Algorithme du gradient

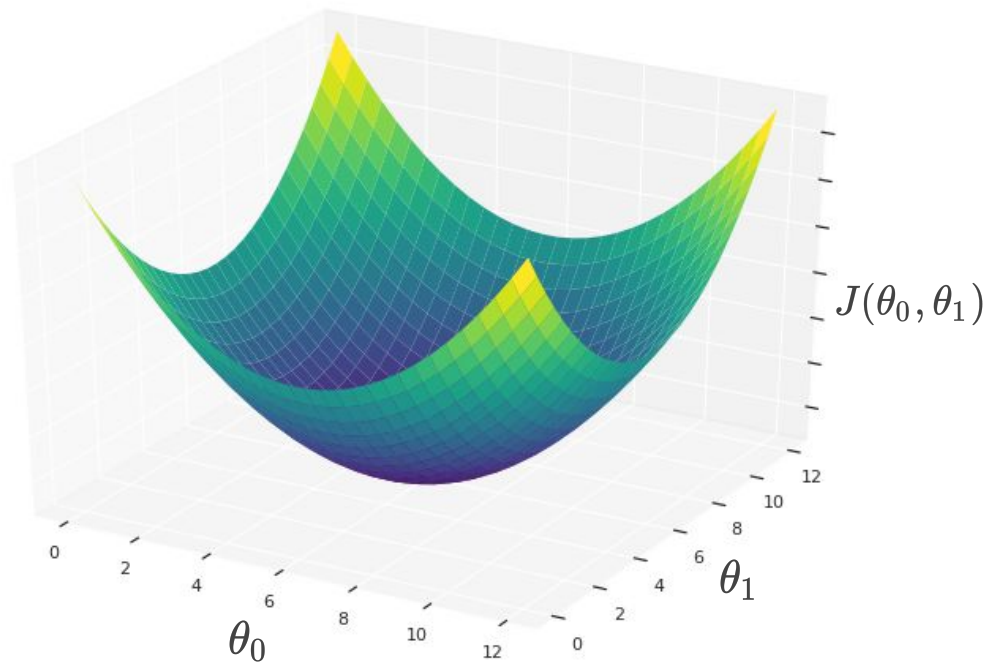
$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

$$\theta_1 := \theta_1 - \alpha \times \text{nombre négatif}$$



Algorithme du gradient

- En régression linéaire, la fonction de coût est une **fonction convexe**
- Forme de **bol**
- La fonction de coût possède un **minimum global**

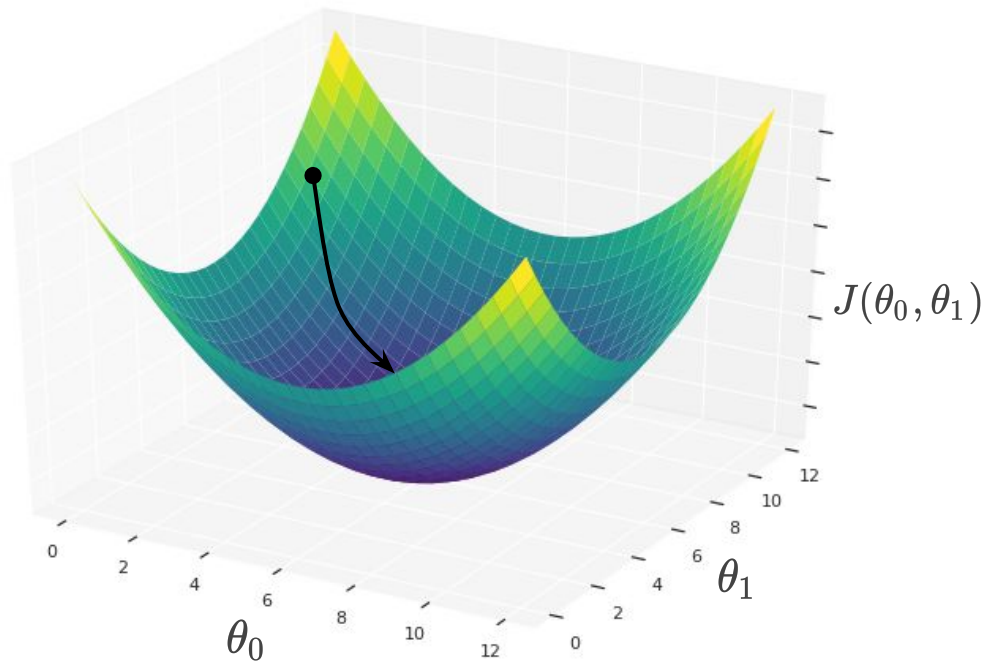


Algorithme du gradient

- Dans le cas de deux paramètres

$$\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

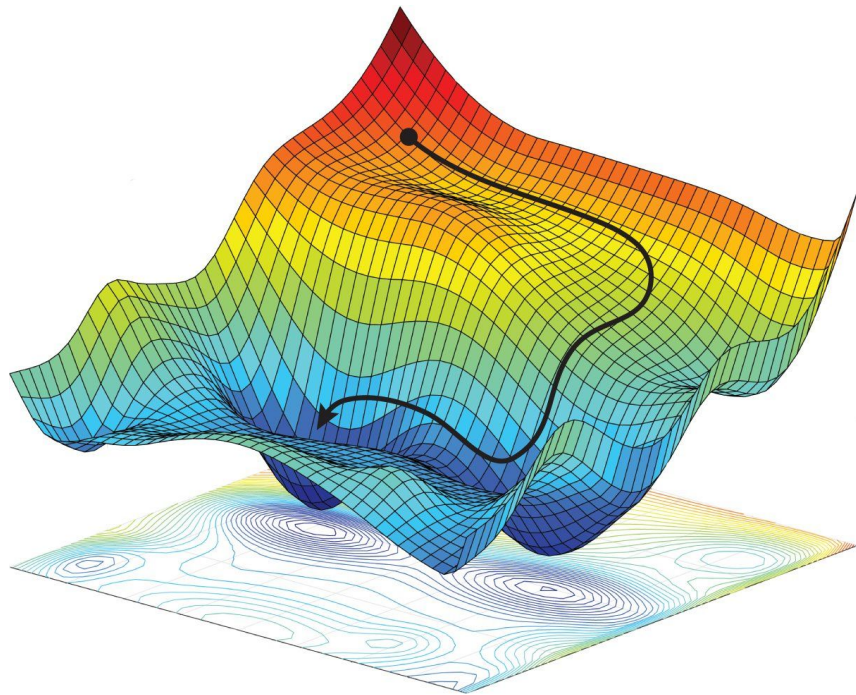
- Commencer avec des **valeurs initiales** pour θ_0 et θ_1 , puis **modifier** les valeurs de θ_0 et θ_1 pour **diminuer** J jusqu'à l'atteinte du **minimum**
- **Algorithme du gradient** (Gradient Descent)



Algorithme du gradient

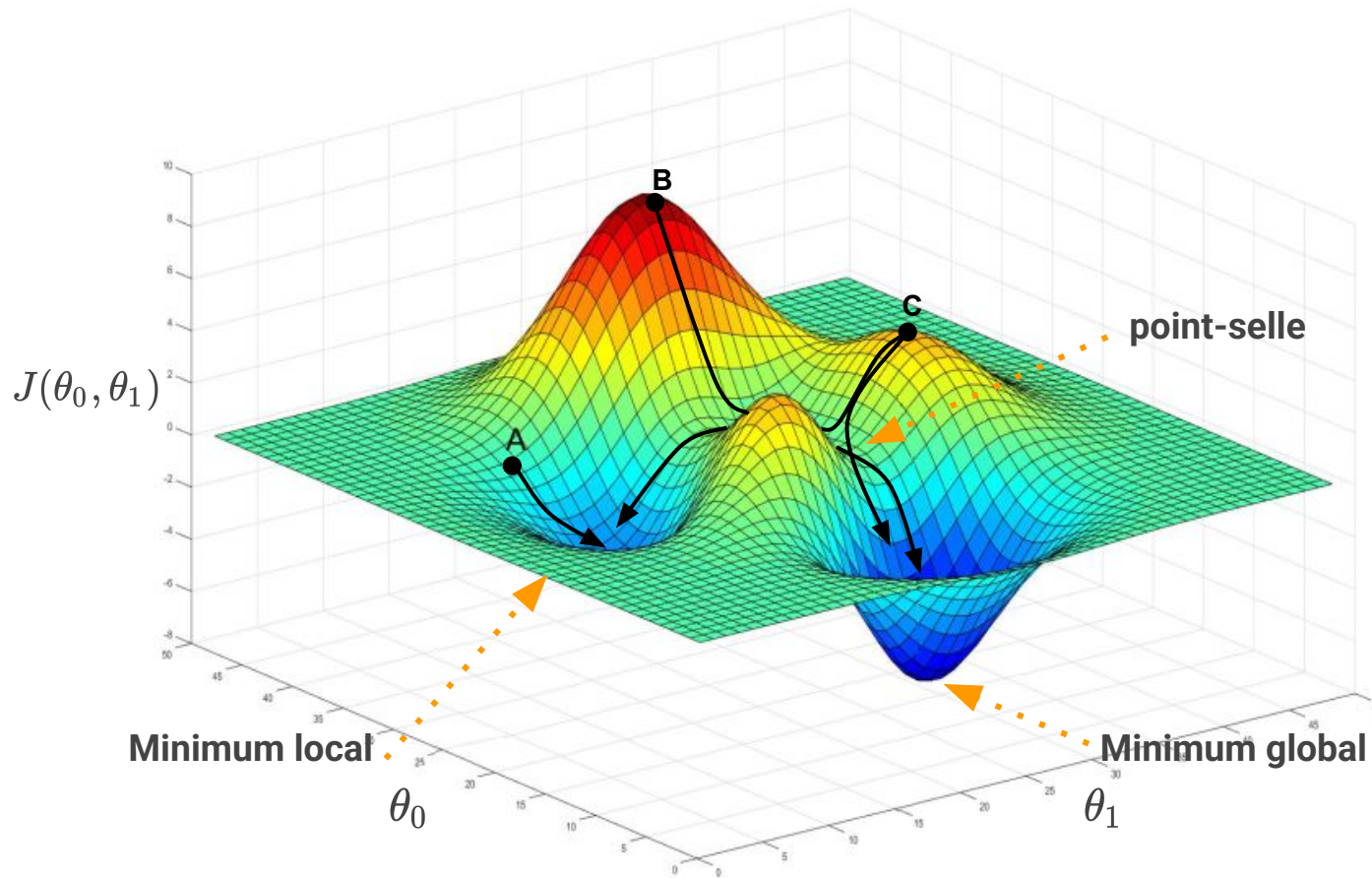
- Dans le cas de plus de deux paramètres

$$\min_{\theta_0, \theta_1, \theta_2, \dots, \theta_n} J(\theta_0, \theta_1, \theta_2, \dots, \theta_n)$$



Stochastic Gradient/Mirror Descent: Minimax Optimality and Implicit Regularization

Algorithme du gradient



Algorithme du gradient

- répéter jusqu'à convergence la mise-à-jour simultanée des paramètres θ_j

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \text{ (pour } j = 0 \text{ et } j = 1)$$

pas / learning rate

dérivée partielle de la fonction de coût

Algorithme du gradient

Bonne implémentation

```
temp0 :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
temp1 :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$   
 $\theta_0$  := temp0  
 $\theta_1$  := temp1
```

Mauvaise implémentation

```
 $\theta_0$  :=  $\theta_0 - \alpha \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$   
 $\theta_1$  :=  $\theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$ 
```

Question

- Supposons que θ_0 et θ_1 soient initialisés respectivement aux valeurs 1 et 4
- θ_0 et θ_1 sont simultanément mis-à-jour avec la règle suivante

$$\theta_j := \theta_j + \alpha \sqrt{\theta_0 \theta_1} \text{ (pour } j = 0 \text{ et } j = 1)$$

- Sachant que la valeur du pas est 1, quelles sont les **nouvelles** valeurs de θ_0 et θ_1 ?

Réponse

- Supposons que θ_0 et θ_1 soient initialisés respectivement aux valeurs 1 et 4
- θ_0 et θ_1 sont simultanément mis-à-jour avec la règle suivante

$$\theta_j := \theta_j + \alpha \sqrt{\theta_0 \theta_1} \text{ (pour } j = 0 \text{ et } j = 1)$$

- Sachant que la valeur du pas est 1, quelles sont les **nouvelles** valeurs de θ_0 et θ_1 ?

$$\begin{aligned} \rightarrow \theta_0 &= 1 + 1 \times \sqrt{1 \times 4} = 3 \\ \theta_1 &= 4 + 1 \times \sqrt{1 \times 4} = 6 \end{aligned}$$

Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- Algorithme du gradient
- Choix du pas
- Régression linéaire simple
- Hyperparamètres
- Lectures et références

Choix du pas

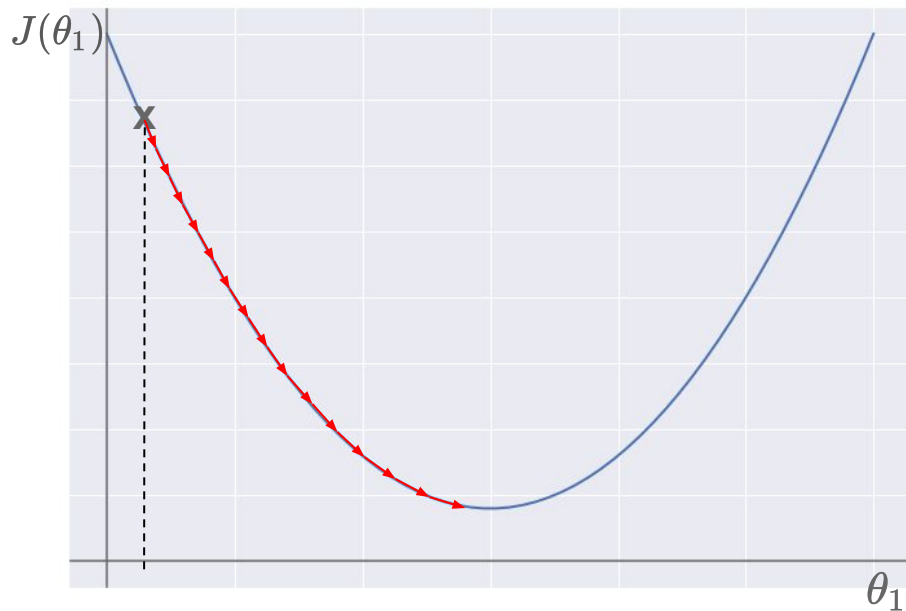
- répéter jusqu'à **convergence** la **mise-à-jour simultanée** des paramètres θ_j

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) \text{ (pour } j = 0 \text{ et } j = 1)$$

pas / learning rate



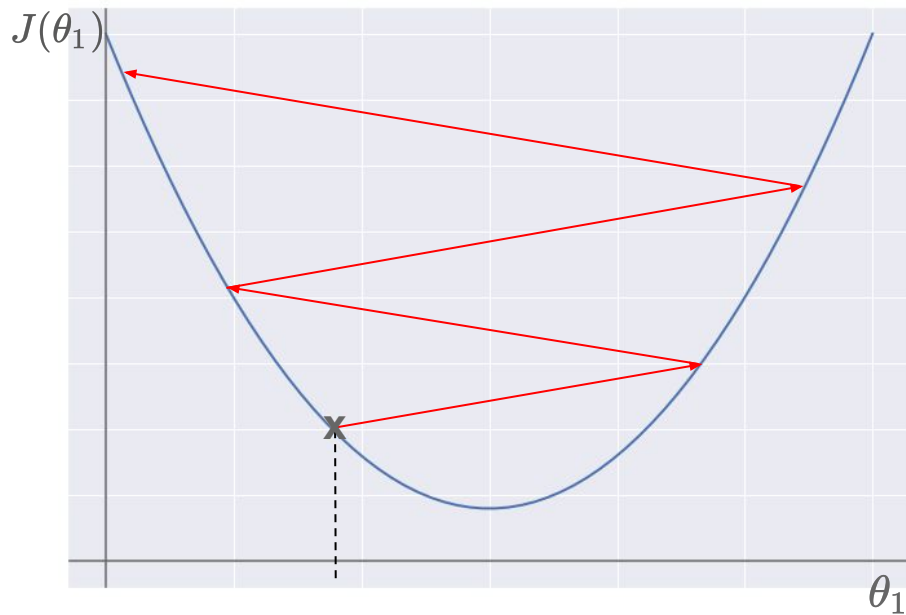
Choix du pas



- Si le **pas** est trop petit, la convergence peut être lente

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

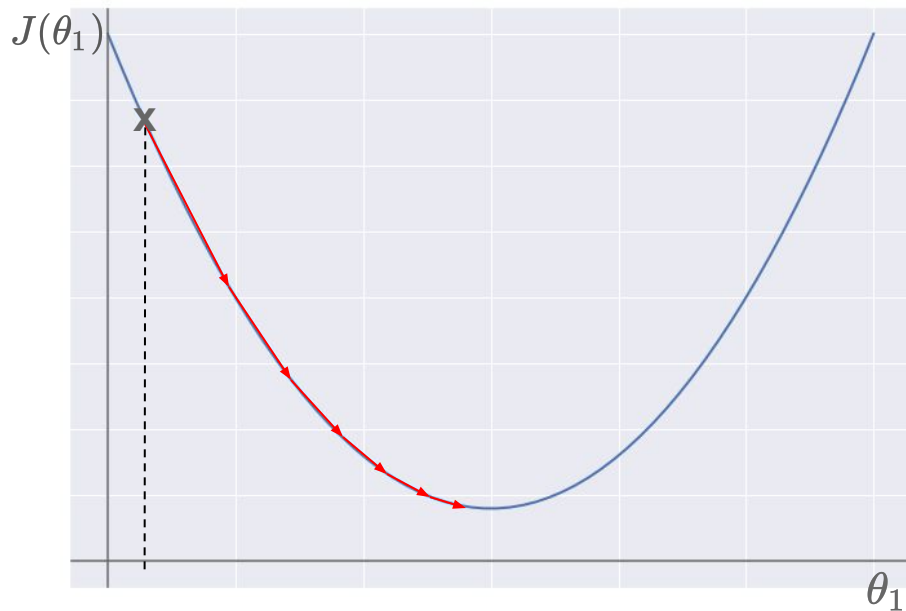
Choix du pas



- Si le **pas** est trop grand, la convergence peut être difficile ou même impossible (oscillations et divergence)

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

Choix du pas

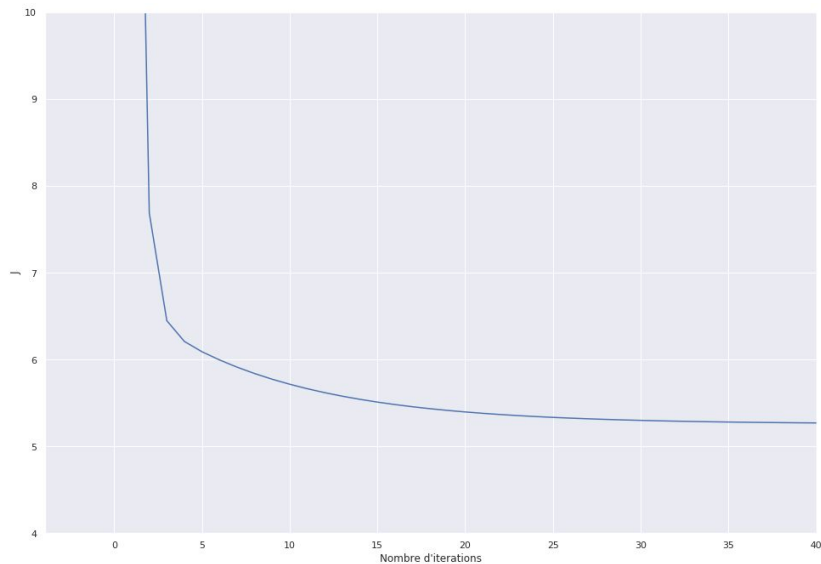


- La convergence vers un minimum local est possible même avec une **valeur du pas fixe**

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

- En régression linéaire, il n'est pas nécessaire de diminuer la valeur du pas au fur et à mesure des itérations (pertinente pour d'autres types de modèles, e.g. Deep Learning, ...)

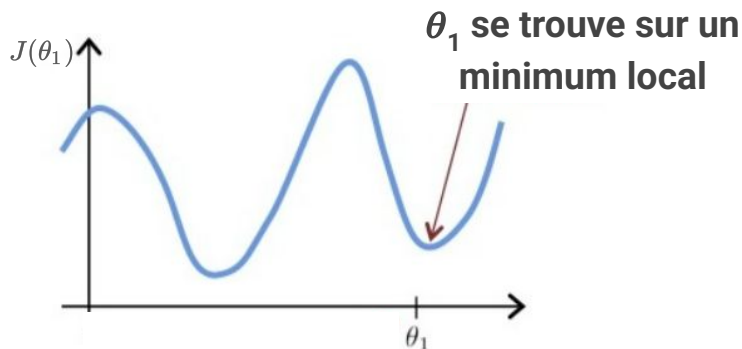
Choix du pas



- Afin de déboguer l'algorithme, il peut être utile d'**afficher la valeur de la fonction de coût au fur et à mesure des itérations**
- Un **test automatique de convergence** peut être mis en place, mais la valeur du seuil peut être difficile à définir

Question

- Supposons que θ_1 soit initialisé sur un minimum local, tel qu'illustré sur la figure suivante. Quel sera l'effet d'une itération de l'algorithme du gradient ?



Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- Algorithme du gradient
- Choix du pas
- **Régression linéaire simple**
- Hyperparamètres
- Lectures et références

Régression linéaire simple

■ Algorithme du gradient

Répéter jusqu'à convergence{

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(pour $j = 0$ et $j = 1$)

}



■ Modèle de régression linéaire simple

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Répéter jusqu'à convergence{ $\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \times x^{(i)}$$

} $\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$

Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- Algorithme du gradient
- Choix du pas
- Régression linéaire simple
- **Hyperparamètres**
- Lectures et références

Paramètres et hyperparamètres

- En apprentissage automatique, un **hyperparamètre** est un paramètre dont la valeur est fixée avant la phase d'entraînement
- Il existe plusieurs techniques permettant de sélectionner les hyperparamètres
- Il existe aussi des **algorithmes de recherche** des hyperparamètres (gridsearchcv, Hyperopt, ...)
- Et il est même possible d'**apprendre** certains hyperparamètres !

Répéter jusqu'à convergence{

$$\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$$

(pour $j = 0$ et $j = 1$)

}

α hyperparamètre

θ_0 et θ_1 paramètres

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

Learning to learn ?

Learning to learn by gradient descent by gradient descent

Marcin Andrychowicz¹, Misha Denil¹, Sergio Gómez Colmenarejo¹, Matthew W. Hoffman¹,
David Pfau¹, Tom Schaul¹, Brendan Shillingford^{1,2}, Nando de Freitas^{1,2,3}

¹Google DeepMind ²University of Oxford ³Canadian Institute for Advanced Research

marcin.andrychowicz@gmail.com

{mdenil,sergomez,mwhoffman,pfau,schaul}@google.com

brendan.shillingford@cs.ox.ac.uk, nandodef Freitas@google.com

Abstract

The move from hand-designed features to learned features in machine learning has been wildly successful. In spite of this, optimization algorithms are still designed by hand. In this paper we show how the design of an optimization algorithm can be cast as a learning problem, allowing the algorithm to learn to exploit structure in the problems of interest in an automatic way. Our learned algorithms, implemented by LSTMs, outperform generic, hand-designed competitors on the tasks for which they are trained, and also generalize well to new tasks with similar structure. We demonstrate this on a number of tasks, including simple convex problems, training neural networks, and styling images with neural art.

<https://arxiv.org/abs/1606.04474>

Sommaire

- Procédure itérative d'apprentissage des paramètres et notation
- Fonction de coût
- Algorithme du gradient
- Choix du pas
- Régression linéaire simple
- Hyperparamètres
- Lectures et références

Lectures et références

- [1] CS229: Machine Learning - Stanford University
- [2] Introduction to Statistical Learning with Applications in Python
- [3] Application des méthodes de régression en statistique - Université Laval